

DATAMATIKER · 2. SEMESTER · EK LYNGBY

Cupcake.

Webshop til Copenhagen Cupcakes — rapport

GRUPPE cupcake-gruppeD · Datamatiker 2. semester E26 · Afleveret 12. november 2026

Navn	Studie-email	GitHub-navn
Nastaran Nazari	cps-nn249@stud.ek.dk	nastanazari1382
Besima Plavotic	bepl1000@stud.ek.dk	BesimaP
Louise Alstad Sverdrup	losv1000@stud.ek.dk	louiseas
Rebecca Støjko Andersen	rean0002@stud.ek.dk	rean0002

GITHUB Cupcake

VIDEO-DEMO Cupcake **(husk lyd & kvalitet på 1080p)**

Indholdsfortegnelse

Indledning 3
- Baggrund 3
- Teknologivalg 3

Krav 4
- Virksomhedens vision 4
- User stories og acceptkriterier 4

Domænemodel og ER-diagram 7
- Domænemodel 7
- ER-diagram 8
- Sådan blev ER-diagrammet til 8

Mockups 10

Navigationsdiagram 13

Implementation 15

Test 17

Status på implementation 18

Proces 19
- Processtyring: GitHub Projects 19
- Git-strategi: branches, pull requests, hvor ofte vi committede 19
- Kodestandarder og Definition of Done 20
- Roller 20
- Læring 21

KAPITEL 01

Indledning





Rapporten beskriver vores løsning på Cupcake-opgaven på 2. semester af Datamatikeruddannelsen. Den er skrevet til en medstuderende på 2. semester, som ikke kender opgaven.

Baggrund

Copenhagen Cupcakes er en virksomhed, der sælger cupcakes. Kunden har brug for, at kunderne kan bestille og betale cupcakes online med valgfri bund og topping og senere hente dem i butikken uden at stå i kø. Administratoren skal kunne have et overblik over alle ordrer og kunder.

Teknologivalg

Teknologi	Version
Java	21
Javalin	7.2.3
Thymeleaf	3.1.5
PostgreSQL / PgAdmin	PostgreSQL 16.2 / pgAdmin 4 (9.19)
JDBC-driver	PostgreSQL JDBC 42.7.8 (HikariCP 7.0.2 som connection pool)
IntelliJ IDEA	2026.2.3
HTML / CSS	HTML5 / CSS3
GitHub	GitHub (Projects, pull requests)
JUnit	6.1.3

KAPITEL 02

Krav

Virksomhedens vision

Copenhagen Cupcakes ønsker en webshop, hvor kunderne frit kan vælge og designe deres egne cupcakes, bestille dem og betale for dem. Webshoppen skal også give bageriet bedre overblik over bestillingerne. I stedet for at kunderne skal stå i kø, skal de kunne bestille og betale på forhånd og blot hente deres ordrer. Det er også en fordel for Copenhagen Cupcakes, fordi de med et overblik over bestillingerne bedre kan planlægge, hvor meget tid og ressourcer de skal bruge. Samlet set skal det give kunden en nem og hurtig oplevelse.

User stories og acceptkriterier

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken og hente min ordre uden at stå i kø.

Acceptkriterier:

- **Givet** at jeg er på forsiden, **når** siden vises, **så** kan jeg vælge mellem alle bunde og toppe med priser fra databasen.
- **Givet** at jeg har valgt bund, top og antal, **når** jeg trykker "Læg i kurv", **så** lægges cupcaken i min kurv.
- **Givet** at jeg ikke er logget ind, **når** jeg trykker "Betal og bestil", **så** sendes jeg til log ind-siden.
- **Givet** at jeg er logget ind og har penge nok, **når** jeg trykker "Betal og bestil", **så** trækkes beløbet fra min saldo, ordren og ordrelinjerne gemmes med stykpris (unit_price), kurven tømmes, og jeg ser en bekræftelse med ordrenummer og ny saldo.
- **Givet** at min saldo er lavere end kurvens pris, **når** jeg trykker "Betal og bestil", **så** vises en fejlbesked, og ordren gemmes ikke.

US-2: Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

Acceptkriterier:

- **Givet** at jeg er på opret konto-siden, **når** jeg skriver en ny email og to ens kodeord og trykker "Opret", **så** oprettes jeg som bruger med rollen 'customer' og saldo 0 kr., og jeg sendes til kurv-siden.
- **Givet** at emailen allerede findes, **når** jeg prøver at oprette en konto med den, **så** vises en fejlbesked, og der oprettes ingen ny bruger.
- **Givet** at de to kodeord ikke er ens, **når** jeg trykker "Opret", **så** vises en fejlbesked.

US-3: Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.

Acceptkriterier:

- **Givet** at jeg har lagt cupcakes i kurven, **når** jeg åbner kurven, **så** ser jeg hver linje med bund, top, antal og pris og den samlede pris for alle linjer.
- **Givet** at jeg har varer i kurven, **når** jeg går til en anden side, **så** er kurven stadig fyldt, og antallet vises ved "Kurv" i topmenuen.

US-4: Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side.

Acceptkriterier:

- **Givet** at jeg er kunde med en konto, **når** jeg logger ind med korrekt email og kodeord, **så** sendes jeg til kurv-siden, og min email vises i topmenuen på alle sider.
- **Givet** at jeg er admin, **når** jeg logger ind, **så** sendes jeg til ordresiden (/admin/ordrer).
- **Givet** at email eller kodeord er forkert, **når** jeg trykker "Log ind", **så** vises en fejlbesked, og jeg er ikke logget ind.
- **Givet** at jeg er logget ind, **når** jeg trykker "Log ud", **så** forsvinder min email fra topmenuen.

US-5: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se, hvad der er blevet bestilt.

Acceptkriterier:

- **Givet** at jeg er logget ind som admin, **når** jeg åbner ordresiden, **så** ser jeg alle ordrer med ordrenummer, kundens email, dato, samlet pris og om ordren er betalt, nyeste øverst.
- **Givet** at jeg er logget ind som kunde, **når** jeg prøver at åbne ordresiden, **så** sendes jeg til forsiden.

US-6: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

Acceptkriterier:

- **Givet** at jeg er logget ind som admin, **når** jeg åbner kundesiden, **så** ser jeg alle kunder med email og saldo og hver kundes ordrer.
- **Givet** at jeg ikke er admin, **når** jeg prøver at åbne kundesiden, **så** sendes jeg til forsiden.

US-7: Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.

Acceptkriterier:

- **Givet** at jeg har flere linjer i kurven, **når** jeg trykker "Fjern" ved en linje, **så** forsvinder kun den linje, og den samlede pris opdateres.
- **Givet** at jeg fjerner den sidste linje, **når** kurven vises, **så** står der "Din kurv er tom", og jeg kan ikke betale.

US-8: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer, fx hvis kunden aldrig har betalt.

Acceptkriterier:

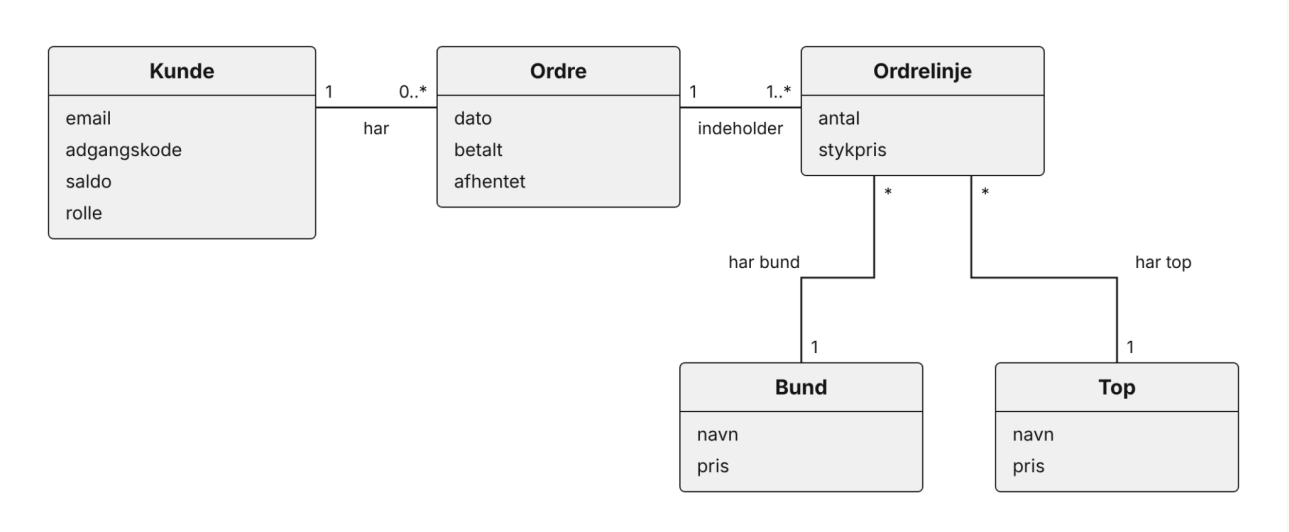
- **Givet** at jeg er admin på ordresiden, **når** jeg trykker "Slet" ved en ordre, **så** forsvinder ordren og alle dens ordrelinjer fra databasen.
- **Givet** at jeg er kunde, **når** jeg prøver at slette en ordre, **så** sker der ingenting, og jeg sendes til forsiden.

KAPITEL 03

Domænemodel og ER-diagram

Domænemodel

Domænemodellen viser begreberne i webshoppen, som kunden og virksomheden taler om dem. Den er på dansk og uden metoder.

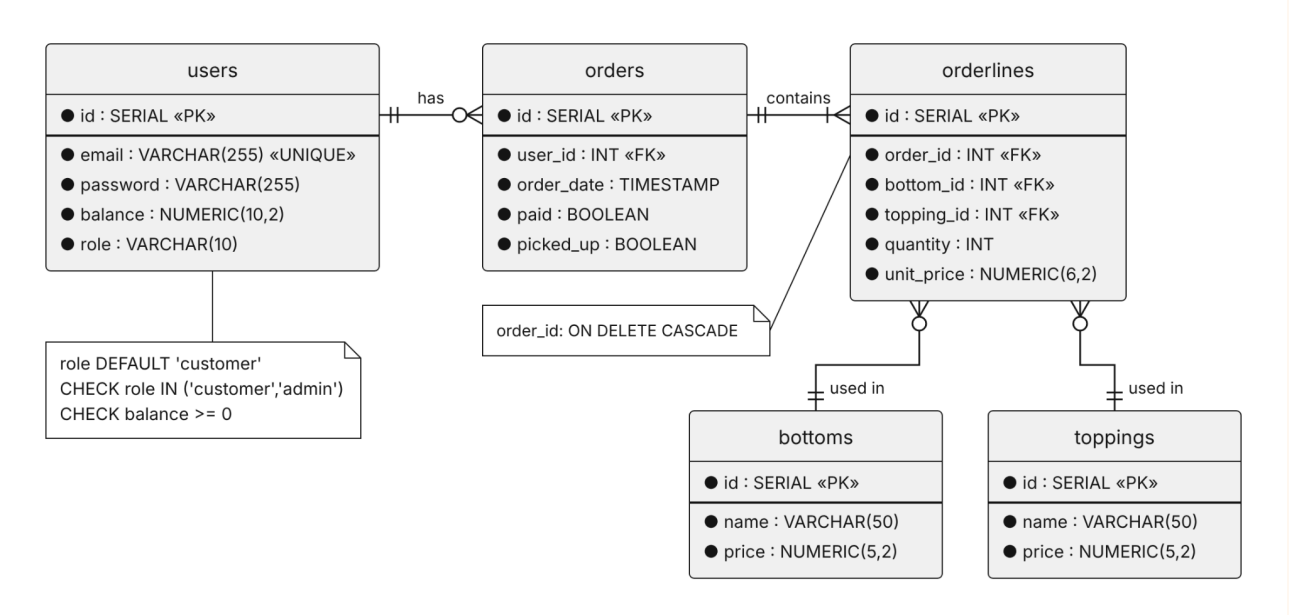


Figur 1: Domænemodel (egen figur).

- **Kunde 1 – 0..* Ordre:** En kunde kan have nul eller flere ordrer, fordi en ny kunde endnu ikke har bestilt noget. En ordre tilhører præcis én kunde.
- **Ordre 1 – 1..* Ordrelinje:** En ordre uden linjer giver ikke mening, så der skal være mindst én.
- **Ordrelinje * – 1 Bund og * – 1 Top:** En ordrelinje er én slags cupcake med præcis én bund, én top og et antal. En bund eller top kan indgå i mange ordrelinjer eller ingen.

Admin er ikke en egen klasse, fordi en admin er en kunde med en anden rolle. Indkøbskurven er heller ikke med, fordi den kun ligger midlertidigt i sessionen, indtil kunden betaler. Bund og Top er to klasser og ikke én “Smag”, fordi de er forskellige ting med hver sin liste og pris, og kunden altid vælger én af hver.

ER-diagram



Figur 2: ER-diagram over databasen (egen figur).

Databasen har fem tabeller: users, orders, orderlines, bottoms og toppings. Alle tabeller har id SERIAL som primærnøgle.

Databasen er i 3. normalform med én bevidst undtagelse. Orders og orderlines er delt op, så kunde og dato ikke gentages for hver cupcake, og navn og pris på bunde og toppe står kun ét sted.

Undtagelsen er unit_price på orderlines. Prisen kunne slås op i bottoms og toppings, men hvis virksomheden ændrer priserne, ville gamle ordrer skifte pris. Derfor gemmer vi stykprisen på købstidspunktet. Totalen regnes ud som quantity × unit_price og gemmes ikke. Opgaven siger 3NF, "medmindre andet giver bedre mening", og historiske priser er et godt eksempel: en kvittering må ikke ændre sig bagefter.

ON DELETE CASCADE står kun på orderlines.order_id, så ordrelinjerne forsvinder, når admin sletter en ordre (US-8). Vi har også constraints som email UNIQUE, CHECK (balance >= 0) og CHECK (quantity > 0), så databasen selv afviser ugyldige data.

Sådan blev ER-diagrammet til

Vi lavede ikke ER-diagrammet i ét hug. Vi startede med et udkast ud fra user stories og rettede det i syv runder ud fra spørgsmål og feedback:

1. Bund og top blev to tabeller, fordi de er forskellige ting med hver sin pris. Rollen blev tekst i stedet for en boolean, så den kan udvides.
2. Vi spurgte, hvad der sker med gamle ordrer, hvis prisen ændres. Derfor gemmer vi stykprisen på ordrelinjen.

3. Vi spurgte, hvad der sker med ordrelinjerne, når admin sletter en ordre. Derfor kom ON DELETE CASCADE på orderlines.order_id.
4. Admin skulle kunne se, om en ordre er hentet, så kunne vi tilføje picked_up.
5. Kolonnenavnene blandede dansk og engelsk. Vi besluttede, at domænemodellen er på dansk, og at database og kode er på engelsk.
6. Efter et review omdøbte vi “pris” til unit_price, så det er tydeligt, at det er stykprisen, og tilføjede CHECK på role og balance.
7. Vi overvejede at kalde fremmednøglen users_id, men beholdt user_id, fordi den peger på én række.

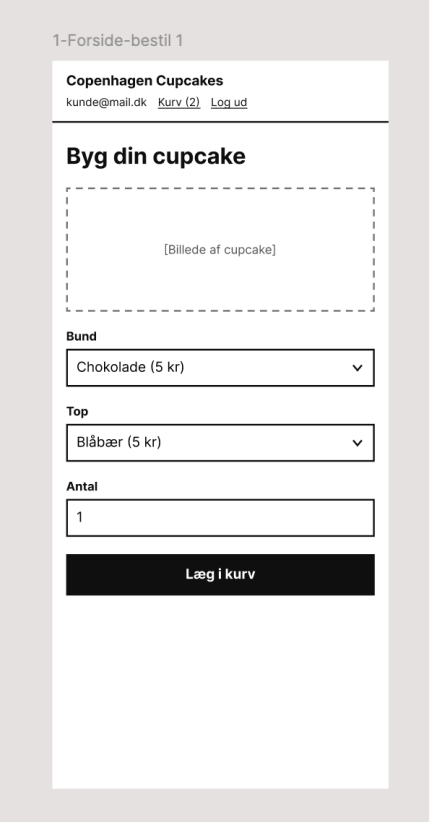
Det betalte sig at stille spørgsmål til designet tidligt. Ændringerne var små at lave i ER-diagrammet, men ville have været store at rette, efter koden var skrevet.

KAPITEL 04

Mockups

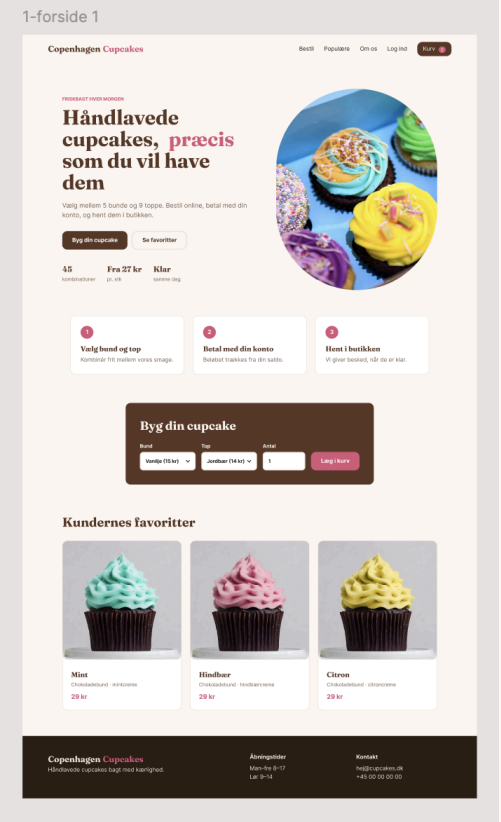
Vi startede med at tegne low-fi mockups i Figma af alle sider i webshoppen. Formålet var, at vi i gruppen skulle blive enige om, hvilke sider der skulle være, og hvad kunden skulle kunne gøre på hver af dem. Det var vigtigt at få gjort, inden vi begyndte at kode, da fejl er nemmere at rette i Figma end i kode, der allerede er skrevet. Derefter lavede vi hi-fi versionerne af de samme sider, hvor vi aftalte farver, typografi og billeder som vi brugte som skabelon for vores HTML og CSS.

Low-fi forside



Figur 3: Low-fi (egen figur).

Hi-fi Forside



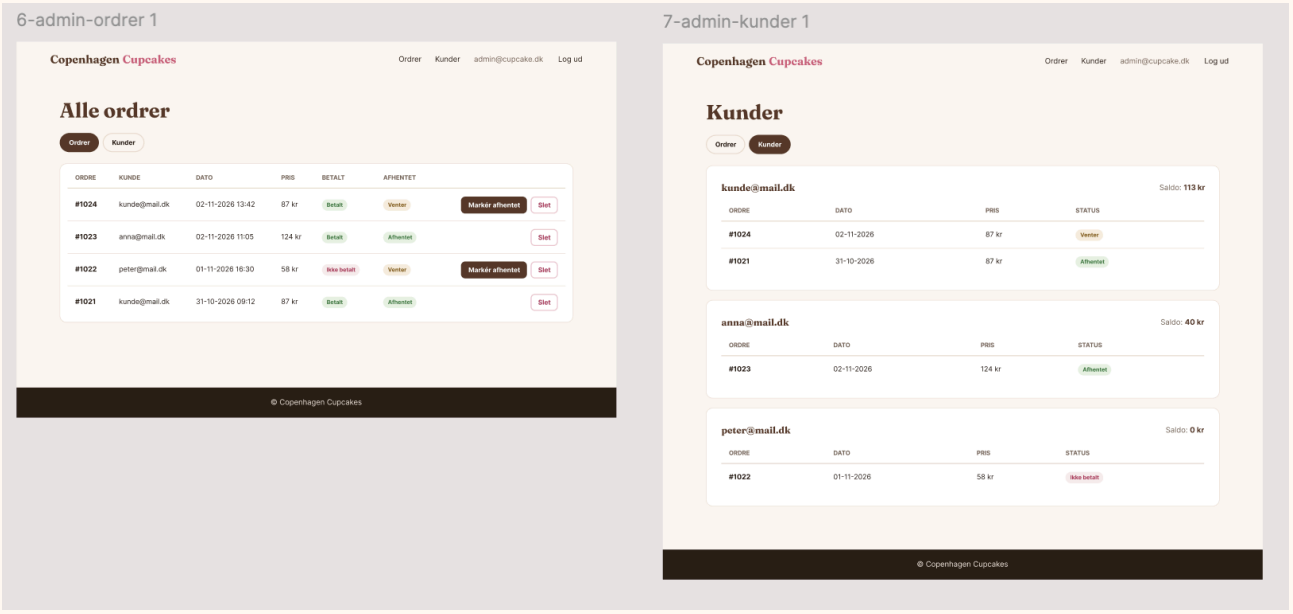
Figur 4: Hi-fi (egen figur).

Alle mockups kan ses i Figma: https://www.figma.com/design/o2OclH3rPqqOVhTMKnXDyF/Cupcake?node-id=2-3

Hvad ændrede sig og hvorfor?

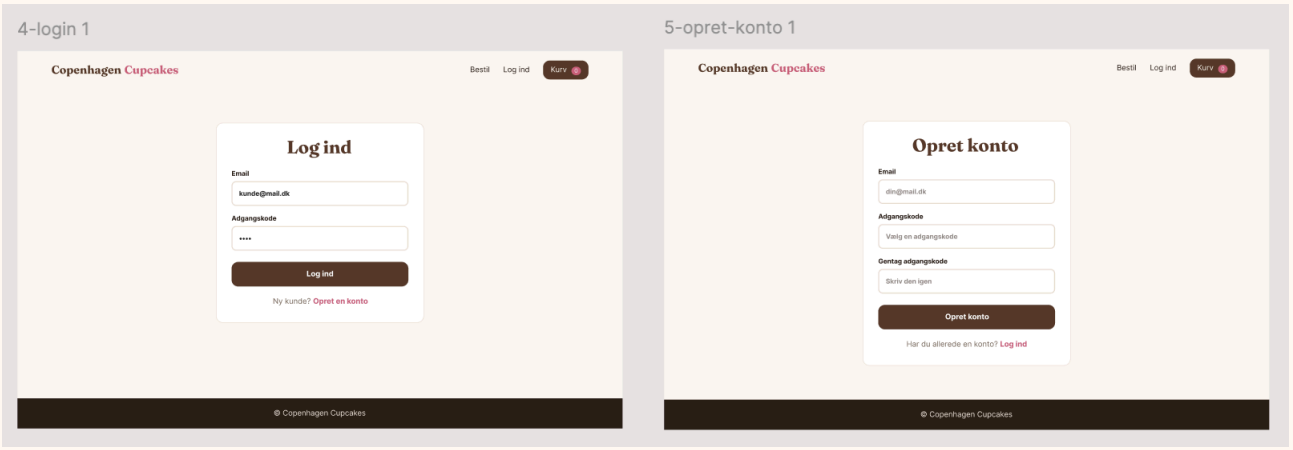
Forsiden er den side der har ændret sig mest. I low-fi består den kun af bestillingsformularen, hvor kunden kan vælge top, bund, antal og lægge i kurven. I hi-fi har vi tilføjet CTA-knapper øverst, 3 korte trin, som forklarer, hvordan man bestiller, og en sektion med kundernes favoritter. Ændringerne skal være med til at gøre siden mere indbydende og give kunderne inspiration før de bestiller. Selve bestillingsformularen er stort set den samme som i low-fi. Før brugte vi opgavens smage og priser fx *"Blåbær (5 kr)"*, men valgte at ændre navne og priser, som passer bedre til et bageri.

Admin sider



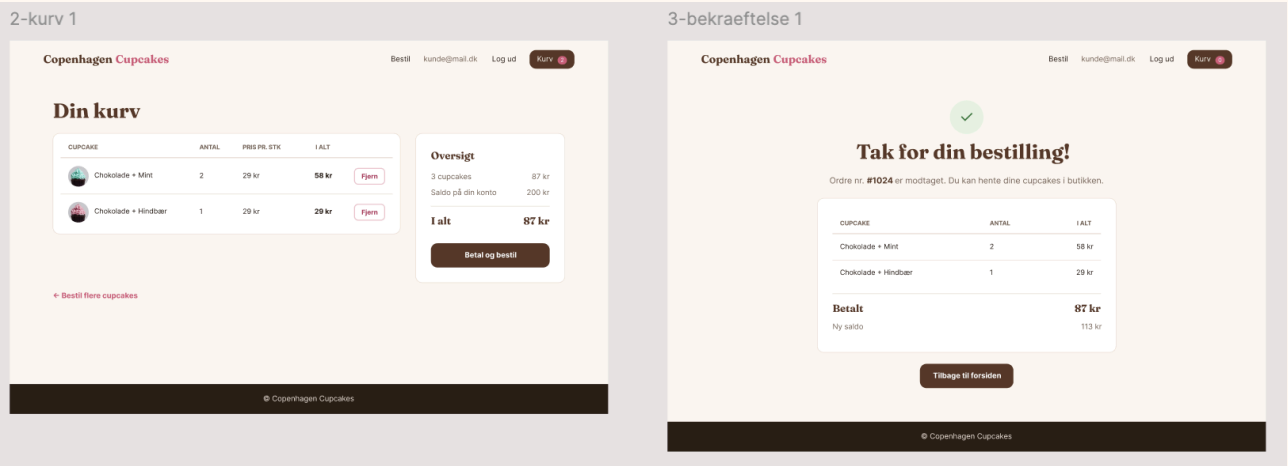
Figur 5: Admin-sider (egen figur).

Log Ind/opret konto



Figur 6: Login/oprettelse af konto (egen figur).

Kurv/bekræftelse



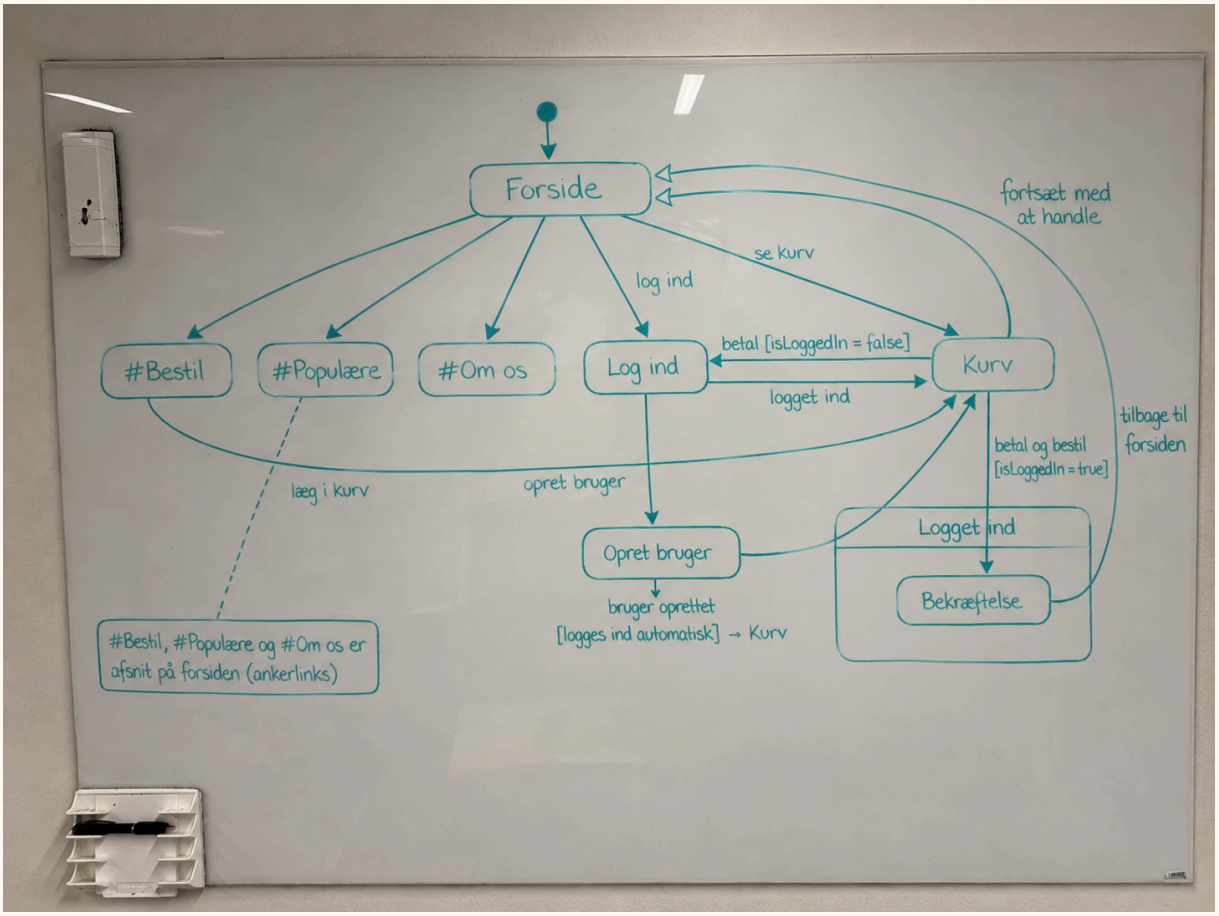
Figur 7: Kurv/Bekræftelse på bestilling (egen figur).

KAPITEL 05

Navigationsdiagram

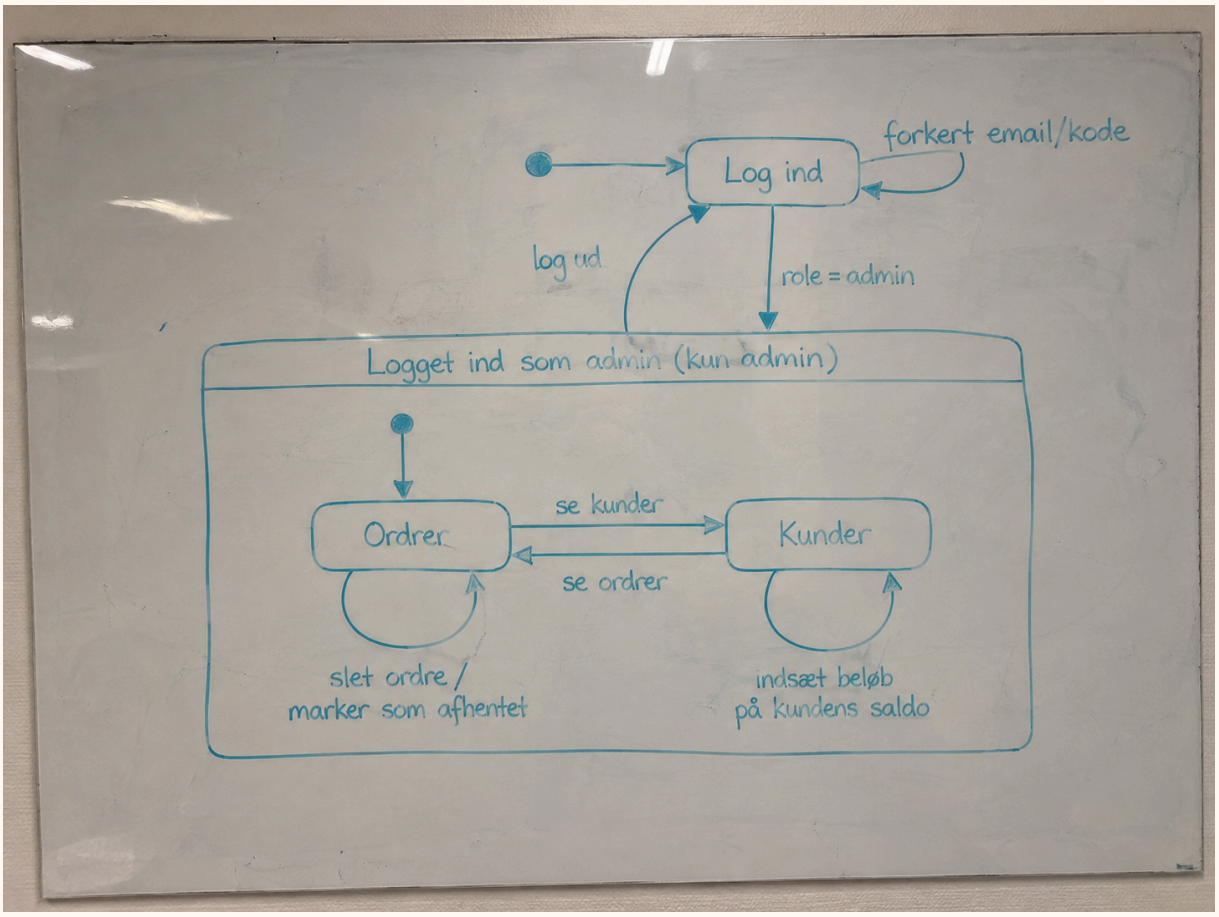
Navigationsdiagrammet viser, hvordan brugeren kan bevæge sig rundt mellem siderne. Vi tegnede det først på tavlen i gruppen, et for kunden og et for admin.

Kunden starter på forsiden, hvor man bygger sin cupcake og lægger den i kurven. Man kan se kurven uden at være logget ind. Når man trykker “Betal og bestil”, bliver man sendt til log ind, hvis man ikke allerede er logget ind. En ny kunde kan oprette en konto og bliver så logget ind automatisk og sendt tilbage til kurven. Efter betaling kommer man til bekræftelsessiden.



Figur 8: Navigationsdiagram for kunden (egen figur).

Admin lander på siden med alle ordrer efter log ind. Herfra kan admin slette ordrer og gå til siden med kunder og deres ordrer.



Figur 9: Navigationsdiagram for admin (egen figur).

Hvorfor kan man se kurven uden at logge ind? Vi ville ikke tvinge kunden til at logge ind, før de havde valgt noget. Det gør det nemmere at handle, og først når der skal betales, skal vi vide, hvem kunden er.

Vi har også tænkt på nogle kendte designprincipper, da vi lavede navigationen.

Jakobs lov siger, at brugere forventer, at en hjemmeside virker som andre hjemmesider, de kender. Derfor står logoet til venstre i toppen, og kurven og log ind står til højre, ligesom i de fleste webshops.

Hicks lov siger, at jo flere valg man får, jo længere tid tager det at vælge. Derfor har menuen kun få punkter, og kunden skal kun vælge bund, top og antal for at bestille.

Fitts' lov siger, at store knapper er nemmere at ramme. Derfor er de vigtigste knapper som "Læg i kurv" og "Betal og bestil" store og mørke, så de er nemme at finde, også på mobil.

KAPITEL 06

Implementation

Det sværeste i en webshop er at håndtere penge sikkert, at holde styr på forbindelsen til databasen og at sikre, at kun admin kan se admin-siderne. Derfor fremhæver vi betalingen i OrderService, vores ConnectionPool og adgangskontrollen. Først et kort overblik over opbygningen.

Opbygning

Koden er delt op i controllere, services og mappere. Controllerne tager imod forespørgsler fra browseren og vælger, hvilken side der skal vises. OrderService indeholder forretningslogikken for kurven og betalingen. Mapperne er de eneste klasser, der snakker med databasen. Opdelingen gør koden lettere at overskue, og vi kunne arbejde på hver sin del uden at komme i vejen for hinanden. Vi har dog kun lavet en service til ordreflowet. UserController og AdminController kalder mapperne direkte, fordi de næsten ikke har nogen logik. Med mere tid ville vi have gjort det ens overalt.

Betaling i OrderService

Når kunden trykker “Betal og bestil”, kalder OrderController metoden placeOrder:

```java
if (cart == null || cart.isEmpty()) {
    throw new DatabaseException("Kurven er tom");
}
double total = calculateTotal(cart);
if (user.getBalance() < total) {
    throw new DatabaseException("Du har ikke nok penge på kontoen");
}
```

Figur 10: Tjek af kurv og saldo i OrderService.placeOrder.

Begge tjek sker, før der skrives noget til databasen. Er kurven tom, eller har kunden ikke penge nok, bliver der ikke gemt noget. Først derefter gemmes ordren, ordrelinjerne og den nye saldo. Controlleren fanger fejlen og viser beskeden på kurvsiden, så kunden kan se, hvad der gik galt. Fordi logikken ligger i servicen og ikke i controlleren, kan vi teste den med unit tests uden en database (se kapitel 7). Stykprisen gemmes på hver ordrelinje som unit_price (se kapitel 3).

Session

Kurven ligger i sessionen som en liste af ordrelinjer, og den indloggede bruger gemmes som currentUser. Når kunden vil betale, tjekker OrderController, om currentUser findes, og sender ellers kunden til log ind-siden.

ConnectionPool som singleton

Forbindelsen til databasen styres af en ConnectionPool med HikariCP. Klassen er lavet efter design patternet singleton: konstruktøren er privat, og man får poolen gennem getInstance(). Den oprettes én gang i Main og sendes videre til controllerne, service og mapperne. Så genbruges forbindelserne i stedet for at åbne en ny hver gang. I getInstance() tjekker vi to gange, om poolen allerede findes, og det andet tjek står inde i en synchronized-blok. Det sikrer, at to tråde ikke laver hver sin pool, hvis de kalder metoden på samme tid.

Adgangskontrol på adminsiderne

Alle routes i AdminController starter med at kalde isAdmin(), som tjekker, at brugeren i sessionen har rollen admin. Hvis ikke, sendes brugeren til forsiden. Det gælder også for sletning af en ordre. Det er ikke nok bare at skjule admin-linket i menuen, for en kunde kunne stadig skrive URL'en selv eller sende en POST-forespørgsel til /admin/ordrer/slet.

Validering af input

Vi tjekker brugerens input, før det bruges. Ved oprettelse af en konto må emailen ikke være tom, og de to kodeord skal være ens. Antallet af cupcakes skal være mindst 1. Databasen afviser desuden ugyldige data med constraints som email UNIQUE og CHECK (balance >= 0).

Refleksion

Hvis vi havde haft mere tid, er der nogle ting, vi ville have gjort anderledes.

Når en kunde betaler, gemmer vi ordren, ordrelinjerne og den nye saldo hver for sig. Hvis der sker en fejl undervejs, kan ordren blive gemt, uden at pengene bliver trukket. Det kunne vi løse med en transaktion, så enten bliver alt gemt, eller også bliver intet gemt.

Saldoen bliver tjekket ud fra den bruger, der ligger i sessionen, og ikke ud fra databasen. Hvis saldoen bliver ændret i pgAdmin, mens kunden er logget ind, bruger vi derfor den gamle saldo, og ændringen bliver overskrevet ved næste køb.

Det hænger sammen med, at admin i dag kun kan give kunder penge direkte i pgAdmin. Vi ville gerne have lavet en funktion på admin-siden, hvor admin kan sætte penge ind på en kundes saldo.

Adgangskoder bliver gemt i klartekst. I et rigtigt system skulle de hashes, fx med BCrypt.

Vi bruger double til beløb i Java, hvilket kan give små afrundingsfejl. BigDecimal ville være bedre.

Til sidst bruger vi DatabaseException til alle fejl, også til fejl som "Kurven er tom", som ikke har noget med databasen at gøre. Det ville være tydeligere med en separat exception til den slags fejl.

KAPITEL 07

Test

Vi har testet webshoppen på tre måder: med unit tests af forretningslogikken, med integration tests af mapperne og med manuelle test af hele flowet ud fra acceptkriterierne.

Unit tests

Unit testene ligger i OrderServiceTest og tester forretningslogikken i OrderService. Der er seks tests. Fire af dem tester udregningen af totalen: en tom kurv giver 0, en kurv der mangler (null) giver 0, én ordrelinje giver stykpris gange antal, og flere linjer giver summen af alle linjer. De sidste to tester betalingen: hvis kunden ikke har penge nok, eller hvis kurven er tom, bliver der kastet en DatabaseException.

Vi valgte at teste OrderService, fordi det er her, de vigtigste regler for webshoppen ligger. Testene kræver ikke en database, fordi vi opretter servicen med new OrderService(null). Det viser samtidig, at tjekkene sker, før databasen bliver brugt. Hvis placeOrder prøvede at gemme noget, ville testen fejle med en anden fejl, fordi der ikke er nogen forbindelse. Det gør testene hurtige og nemme at køre.

Integrationstests

Mapperne testes med integrationstests mod et separat schema i databasen, der hedder test, så vores rigtige data ikke bliver ændret. Klassen TestDatabase opretter tabellerne én gang før testene i hver testklasse (@BeforeAll) og nulstiller testdata før hver test (@BeforeEach), så hver test starter fra samme udgangspunkt.

Vi har 16 integrationstests: 6 for UserMapper, 2 for BottomMapper, 2 for ToppingMapper og 6 for OrderMapper. De tester både det normale tilfælde og fejl. For eksempel tester vi, at login med forkert kodeord giver en fejl, at man ikke kan oprette en bruger med en email, der allerede findes, og at en ny bruger får rollen customer og saldo 0. For ordrer tester vi, at ordrelinjerne også bliver slettet, når en ordre slettes (ON DELETE CASCADE), og at en gammel ordre bruger den gemte unit_price.

Sammen med de seks unit tests har vi 22 tests i alt, og de består alle.

Manuel test

Vi testede hele flowet manuelt i browseren ud fra acceptkriterierne for hver user story. Som kunde testede vi at oprette en konto, logge ind, bygge en cupcake, lægge den i kurven, fjerne den igen, betale og se bekræftelsen. Vi testede også fejl tilfældene, fx at betale uden at være logget ind og at betale med for lav saldo. Som admin testede vi at se alle ordrer, slette en ordre og se kunderne og deres ordrer. Vi testede også, at en kunde bliver sendt til forsiden, hvis den prøver at åbne admin siderne. Det svarer til en brugeraccepttest (UAT), hvor vi tjekker, at systemet gør det, kunden har bedt om.

Hvornår testede vi

Hver user story blev testet manuelt af den, der lavede den, og igen af den, der reviewede pull requesten. Unit testene blev skrevet sammen med OrderService, mens integrationstestene af mapperne blev lavet til sidst, da koden var færdig. Vi testede ikke vores mockups med brugere uden for gruppen.

Refleksion

Unit testene fanger fejl i udregningen hurtigt, men de tester ikke, om data bliver gemt rigtigt. Derfor er integrationstestene et vigtigt supplement. Ingen af testene dækker dog controllerne eller HTML-siderne. Det så vi, da betalingen crashede efter en merge, fordi bekræftelsessiden manglede i repoet (se kapitel 9). Næste gang vil vi lave integrationstestene løbende i stedet for til sidst og altid teste hele flowet, efter en pull request er merget.

KAPITEL 08

Status på implementation

User story	Status	Bemærkning
US-1	Færdig	Ordren, ordrelinjerne og saldoen gemmes hver for sig, ikke i én transaktion. Går der noget galt midt i, kan der opstå en halv ordre.
US-2	Færdig	Opretter en kunde med rollen customer og saldo 0, logger kunden ind automatisk og sender dem til kurven. Der vises en fejl, hvis emailen findes, eller hvis de to kodeord ikke er ens.
US-3	Færdig	Kurven ligger i sessionen som en liste af ordrelinjer. Siden viser hver linje med antal og pris samt den samlede pris.
US-4	Færdig	Både kunder og admin logger ind med email og kodeord. Brugeren gemmes i sessionen som currentUser, og emailen vises i toppen af siden. Adgangskoder gemmes i klartekst. I en rigtig webshop ville vi hashe dem.
US-5	Færdig	Admin får en oversigt over alle ordrer med ordrenummer, kundens email, dato, pris og om ordren er betalt. Kun admin kan åbne siden.

User story	Status	Bemærkning
		Ud over kravene har vi tilføjet feltet picked_up, så admin på kundesiden kan se, om en betalt ordre venter på at blive hentet. En mulig udvidelse er en knap, så admin kan markere ordren som hentet.
US-6	Færdig	Siden “Kunder” er kun for admin. Systemet henter alle kunder og finder hver kundes ordrer i databasen, og viser dem samlet på ét kort pr. kunde.
US-7	Færdig	Hver linje i kurven har en “Fjern”-knap. Når man trykker på den, forsvinder kun den linje, og den samlede pris regnes ud igen. Fjernes den sidste linje, står der “Din kurv er tom”, og betalingsknappen skjules.
US-8	Færdig	Hver række på ordresiden har en “Slet”-knap, som sender ordrens id til serveren. Serveren sletter ordren i databasen, og siden vises igen uden den. Fordi orderlines har (ON DELETE CASCADE) på (order_id), slettes ordrelinjerne automatisk sammen med ordren.

KAPITEL 09

Proces

Processtyring: GitHub Projects

Vi brugte GitHub Projects som vores tavle. Hver user story blev oprettet som et issue med acceptkriterier og en liste med tasks, fx US-7 “Fjern ordrelinje fra kurven”. Issuet blev flyttet gennem kolonnerne To do, Ready, In progress, In review og Done. Hvert issue blev tildelt en person (assigned), så alle kunne se, hvem der arbejdede på hvad. Vi prioriterede de første seks user stories først, som kunden bad om, og tog derefter de sidste.

Git-strategi: branches, pull requests, hvor ofte vi committede

Vi arbejdede med én branch pr. user story, fx US2-Opret-konto og US6-admin, ud fra master. Vi lavede ikke ændringer direkte på master. Vi arbejdede i par, hvor den ene lavede backend og den anden

frontend. Hver lavede sin egen sub-branch ud fra user storyens branch, fx US4-backend og US4-frontend. De blev først merget ind i user storyens branch og derefter samlet ind i master med én pull request.

Når en user story var færdig, oprettede vi en pull request med en kort beskrivelse af ændringerne og en test og skrev “Closes #nummer”, så issuet blev lukket automatisk. Derefter læste en anden i gruppen pull requesten igennem, før den blev merged med “squash and merge”, så hver user story blev til ét samlet commit på master. Før vi startede på en ny opgave, hentede vi den nyeste master ind i vores branch, så vi undgik store konflikter. Vi committede og pushede dagligt, så vi alle kunne arbejde videre dagen efter.

Efter en kode gennemgang delte vi rettelserne op i små pull requests, én pr. opgave, fx én til CSS-navne og én til filnavne. Så var hver pull request let at gennemgå, og vi undgik konflikter.

Kodestandarder og Definition of Done

Vi aftalte, at al kode skrives på engelsk: Java, database, CSS-klasser og kommentarer. URL’erne er på dansk (fx /kurv og /opret-konto), fordi de ses af kunden, og domænemodellen er på dansk, fordi den beskriver virksomhedens begreber. Hver metode har en kort kommentar, der forklarer, hvad den gør.

Vi brugte opgavens Definition of Done. En user story var først færdig, når den var implementeret, kunne compile og køre, var testet, var reviewet af et andet gruppemedlem, var pushet til GitHub, opfyldte acceptkriterierne og ikke havde kendte alvorlige fejl.

Roller

Vi fulgte opgavens tre roller: Scrum Master, Product Owner og Tech Lead. Rollerne blev trukket lod den 3. november og skiftede hver uge. Vi er fire, så to delte rollen som Scrum Master.

Uge 1 (2.–6. november): Scrum Master Rebecca og Besima, Product Owner Louise, Tech Lead Nasta.
Uge 2 (9.–12. november): Scrum Master Nasta og Louise, Product Owner Rebecca, Tech Lead Besima.

Scrum Master sørgede for, at vi skrev sammen ca. kl. 9 og kl. 13 om, hvor langt hver især var. Når nogen lavede en pull request, skrev de det ud til hele gruppen og tilføjede flere reviewere, så vi ikke sad og ventede på én bestemt person. Tech Lead holdt styr på kodestandarden, fx at al kode og alle kommentarer var på engelsk. Et eksempel er, da Besima som Tech Lead gennemgik koden og fandt danske navne, som vi derefter rettede til engelsk.

Vi arbejdede i par på tværs af gruppen, hvor den ene lavede backend og den anden frontend, og vi byttede hver dag, så alle har prøvet begge dele.

Fordi vi kun var fire, og alle kodede, blev rollerne ikke så skarpt adskilt. Alle tog ansvar for tavlen og for review. Rollerne hjalp mest ved at give én person ansvaret for at huske det.

Læring

Det var godt, at vi lavede ER-diagrammet om flere gange, før vi begyndte at kode. Hver gang fandt vi en fejl eller noget, der manglede, fx stykprisen på ordrelinjen. Det var meget nemmere at rette på papiret end i koden bagefter. Det hjalp også, at vi byttede mellem backend og frontend hver dag, så alle kan forklare hele løsningen.

Næste gang vil vi aftale sprog og navngivning, før vi skriver den første linje kode. Vi besluttede først undervejs, at koden skulle være på engelsk, og måtte derfor bruge tid på at omdøbe CSS-klasser og filnavne bagefter.

Vi vil også teste hele flowet, før en pull request bliver merget. En gang crashede betalingen efter en merge, fordi bekræftelsessiden manglede i repoet. Det stod i vores Definition of Done, at koden skulle være testet, men vi fulgte det ikke altid.